

SafeLine WAF Protection Lab

A controlled DVWA assessment with origin isolation, repeatable attack comparisons, WAF telemetry correlation, and policy tuning.

4 vulnerability classes	3 blocked in sampled tests	1 policy-tuning sequence	28 reviewed screenshots
-----------------------------------	--------------------------------------	------------------------------------	-----------------------------------

Author: Anbu **Test window:** 30 September-1 October 2026 **Environment:** Isolated VMware Fusion lab

Executive summary

The lab confirmed exploitable SQL injection, reflected XSS, command injection, and local file inclusion at the deliberately vulnerable DVWA origin. SafeLine blocked SQL injection, XSS, and file inclusion through the protected route and recorded supporting attack events.

Command injection initially reached DVWA. The evidence sequence later shows a CMD Inj Strict setting and blocked requests. This supports policy tuning as the working remediation; conflicting version labels and incomplete request-level policy records limit causal attribution.

Architecture and test logic



- **Isolation:** DVWA listened only on Ubuntu loopback; direct Kali access to port 8081 failed.
- **Baseline:** a temporary SSH local forward exposed the origin only on Kali loopback.
- **Protected route:** dvwa . lab resolved to SafeLine, which proxied to the origin.
- **Correlation:** prevention claims required both a client-side block and matching SafeLine telemetry.

Validated results

Test	Origin observation	Protected observation	Assessment
SQL injection	Multiple user records	Access Forbidden; Blocked log	Prevented
Reflected XSS	Browser alert executed	Access Forbidden; Blocked log	Prevented
Command injection - initial	www-data returned	www-data returned; Audited	Allowed
Command injection - later test	www-data returned	www-data returned; Audited	Allowed
Command injection - Strict	www-data returned	Access Forbidden; Blocked log	Prevented
Local file inclusion	/etc/passwd records	Access Forbidden; Blocked log	Prevented

How to interpret the result

- A SafeLine log entry marked **Audited** proves detection, not prevention.
- A software upgrade and an enforcement policy are independent controls.
- The WAF is defense in depth; application-side fixes remain the primary remediation.

Environment

Ubuntu	24.04.5 LTS, ARM64, ~6 GiB
Kali	Rolling 2026.2, ARM64
WAF	UI 9.1.0-lts; management CLI 9.4.2
Target	DVWA Low + MariaDB 10
Proxy	Burp Suite Community 2026.8

Evidence - SQL injection and reflected XSS

Each prevention claim combines an origin success, a protected Access Forbidden response, and matching SafeLine telemetry. The report repository contains all three images for each test.

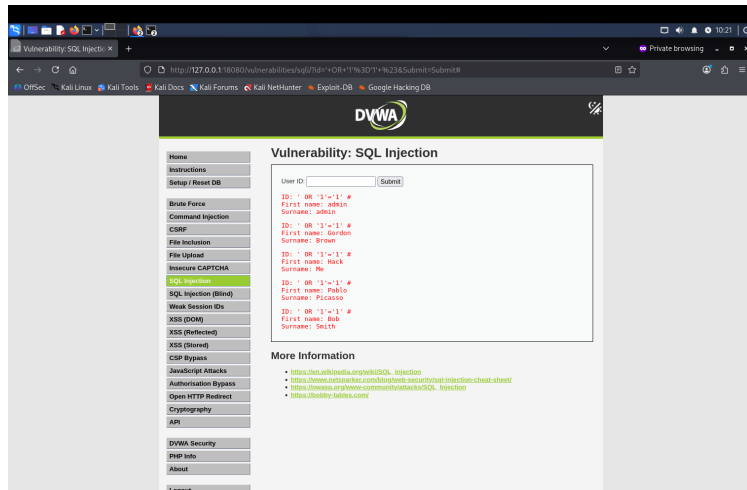


Figure 1. SQL injection at the origin returned multiple user records.

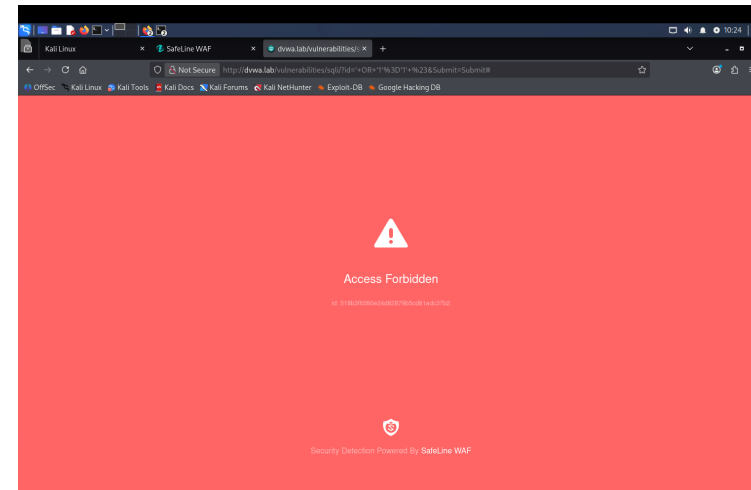


Figure 2. The same test through SafeLine received Access Forbidden.

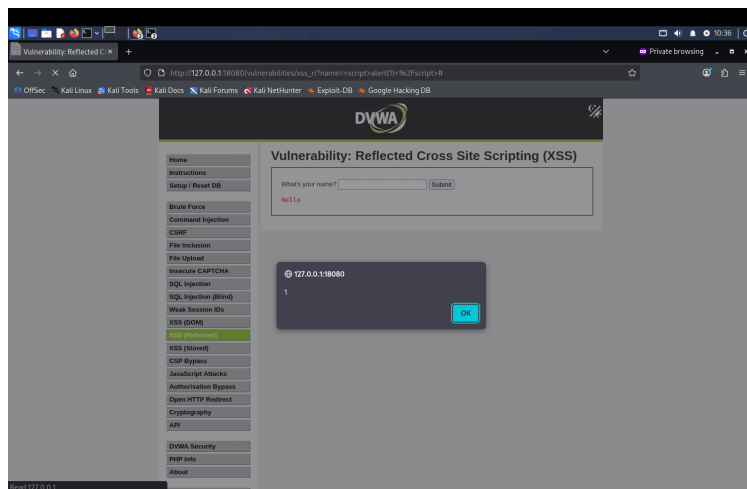


Figure 3. Reflected XSS executed alert(1) at the origin.

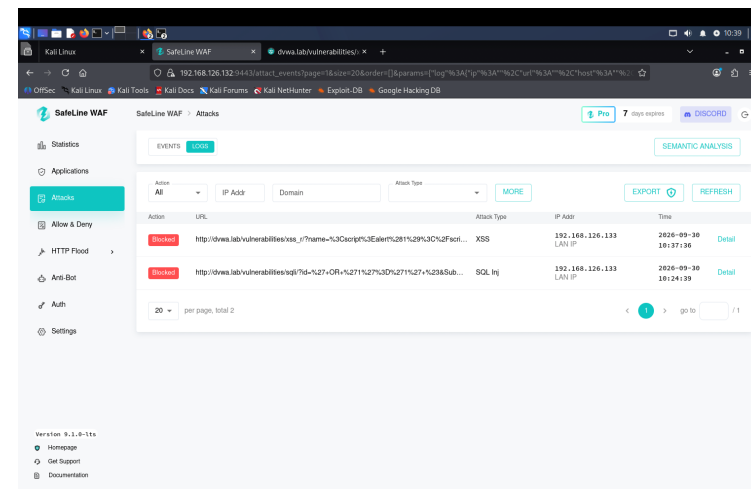


Figure 4. SafeLine recorded the protected XSS request as Blocked.

Key finding - command injection policy gap

The 127.0.0.1; whoami proof returned www-data at the origin and through SafeLine. Later evidence shows Strict policy and blocks. Logs contain both audited and blocked requests; exact version, policy, and request-ID correlation require a synchronized retest.

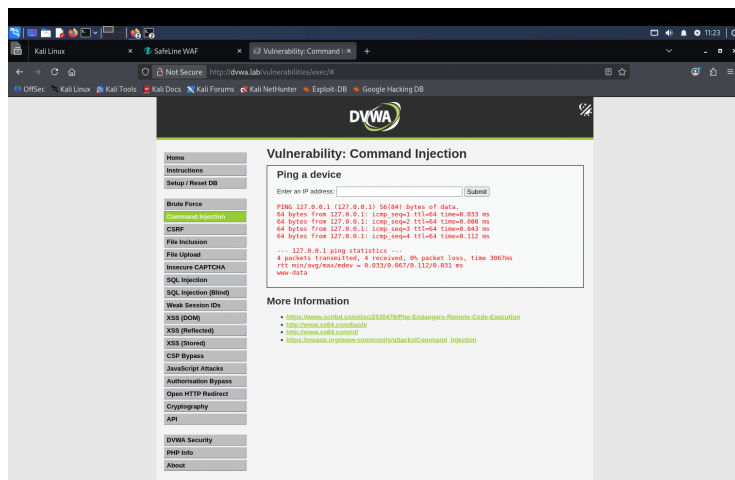


Figure 5. A later protected-route capture still shows command execution.

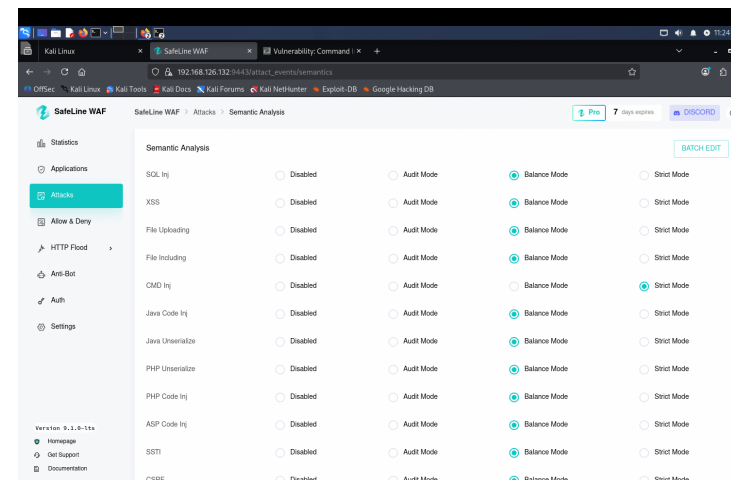


Figure 6. Command-injection semantic policy is shown as Strict.

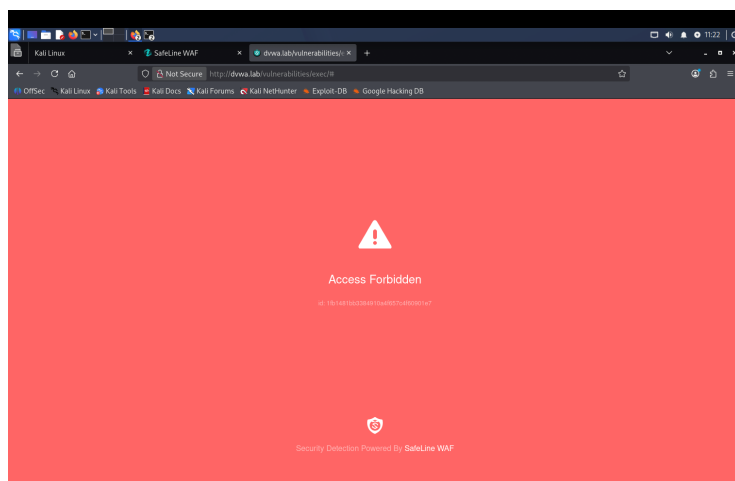


Figure 7. The repeated protected request was blocked.

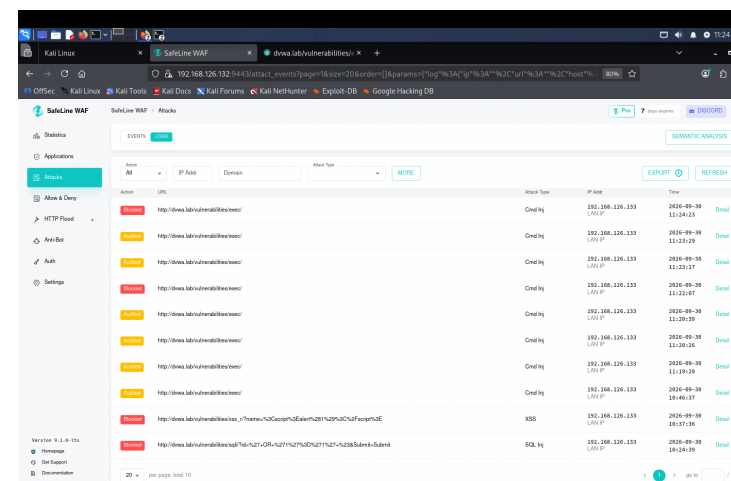


Figure 8. SafeLine records show audited and blocked command-injection events.

Evidence - file inclusion and proxy capture

The file-inclusion comparison used a read-only proof inside the DVWA container. Burp then captured an ordinary cookie-free request to validate the interception path without publishing an authenticated session.

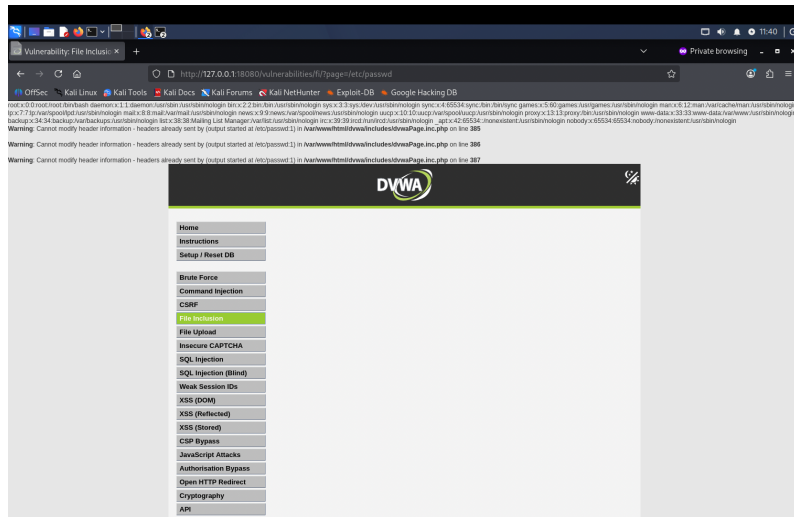


Figure 9. The origin included /etc/passwd and displayed container account records.

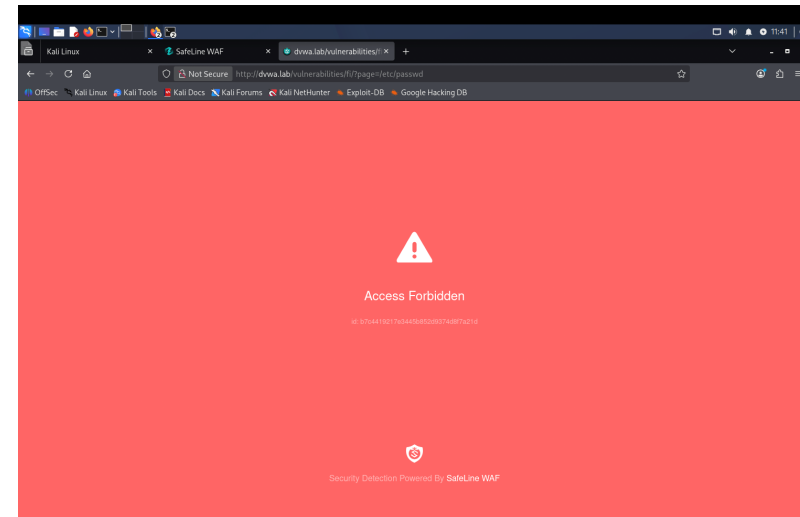


Figure 10. SafeLine denied the protected file-inclusion request.

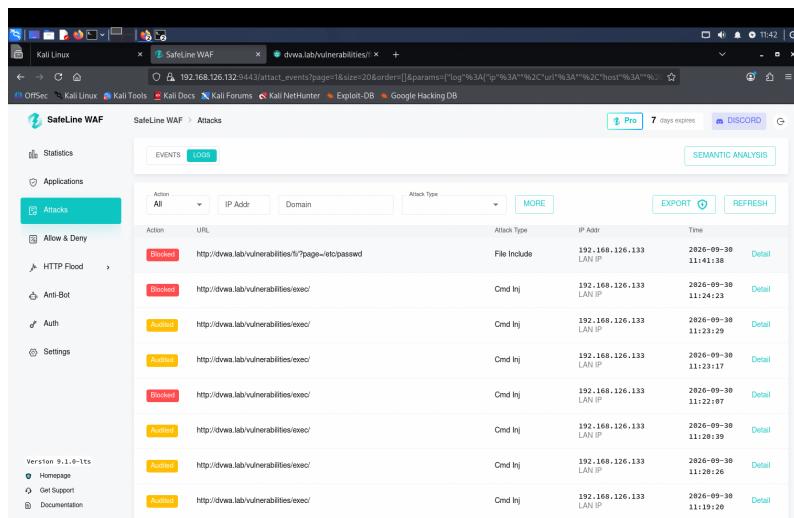


Figure 11. SafeLine classified the request as File Include and Blocked.

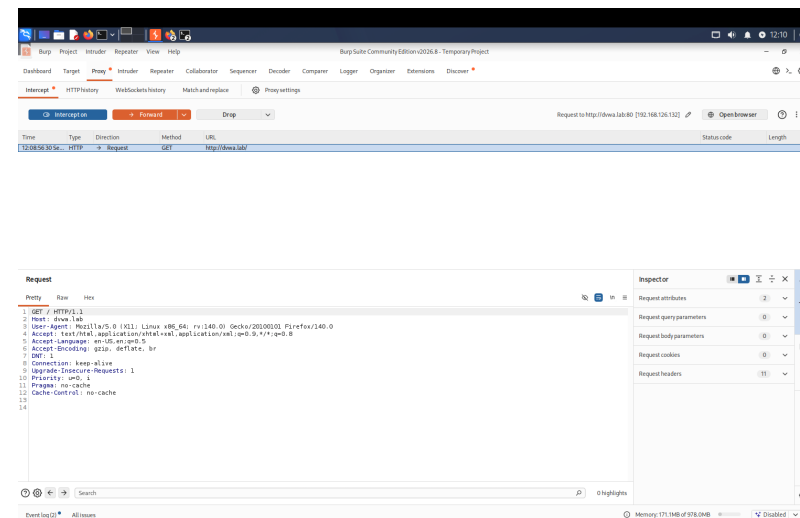


Figure 12. Burp intercepted GET / for dwa.lab with zero request cookies.

Recommendations

- **Fix the application:** use parameterized queries, context-aware output encoding, strict input allowlists, non-shell process execution, and fixed server-side file mappings.
- **Retain origin isolation:** keep DVWA bound to loopback and preserve the host-only network boundary.
- **Keep command injection in Strict:** monitor false positives and stage future policy changes from audit to block.
- **Validate after upgrades:** repeat the same test matrix after SafeLine, application, or policy changes.
- **Protect administration:** restrict the SafeLine management interface to the administrative network and use strong authentication.

Limitations

- One representative payload was used per vulnerability class; this was a control validation rather than exhaustive bypass testing.
- Authentication controls, HTTPS termination, denial of service, persistence, container escape, and post-exploitation were outside scope.
- The dashboard shows 9.1.0-lts while the CLI reports 9.4.2; same-instance correlation is unverified.
- Times were not synchronized across components. Block-page IDs were not joined to event-detail records. Benign regression tests and pinned image digests remain missing.

Conclusion

The evidence demonstrates four origin vulnerabilities and WAF blocks for sampled requests. The command-injection sequence highlights why detection, blocking, software version, and policy state must be verified separately. Strict is the documented remediation setting; stronger causal attribution needs a synchronized retest.

Supporting material

The repository includes the complete 28-image evidence set, SHA-256 manifest, machine-readable CSV, Docker Compose configuration, setup guides, command rationale, troubleshooting notes, and automated portfolio validation.